

Assignment 02

Name : Md Aminul Islam Rony

Roll : ID25M804

Q1: Preprocessing and Inverted Index

After Tokenization and Stopword Removal (stopwords: {the, in, our, and, that, for}):

Document	Tokens After Processing
d1	star, solar, system, provides, heat, light
d2	Hollywood, star, walked, red, carpet, movie, premiere
d3	Astronomers, observe, distant, stars, galaxies, using, telescopes

Inverted Index:

Term	Docs
star	d1, d2
solar	d1
system	d1
provides	d1
heat	d1
light	d1
Hollywood	d2
walked	d2
red	d2
carpet	d2
movie	d2
premiere	d2
Astronomers	d3
observe	d3
distant	d3
stars	d3
galaxies	d3
using	d3
telescopes	d3

Q2: TF-IDF Term-Document Matrix

N = 3 documents. Only terms appearing in more than one document affect IDF meaningfully. For clarity, I am showing only terms present in at least one document.

Term Frequencies (TF = count / doc_length):

Term	d1 (len=6)	d2 (len=7)	d3 (len=7)
star	$1/6 = 0.167$	$1/7 = 0.143$	0
light	$1/6 = 0.167$	0	0
heat	$1/6 = 0.167$	0	0
solar	$1/6 = 0.167$	0	0
system	$1/6 = 0.167$	0	0
provides	$1/6 = 0.167$	0	0
Hollywood	0	$1/7 = 0.143$	0
walked	0	$1/7 = 0.143$	0
red	0	$1/7 = 0.143$	0
carpet	0	$1/7 = 0.143$	0
movie	0	$1/7 = 0.143$	0
premiere	0	$1/7 = 0.143$	0
Astronomers	0	0	$1/7 = 0.143$
observe	0	0	$1/7 = 0.143$
distant	0	0	$1/7 = 0.143$
stars	0	0	$1/7 = 0.143$
galaxies	0	0	$1/7 = 0.143$
using	0	0	$1/7 = 0.143$
telescopes	0	0	$1/7 = 0.143$

Inverse Document Frequencies ($IDF = \log_2(N / df)$):

Term	df	$IDF = \log_2(N / df)$
star	2	$\log_2(3/2) = 0.585$
all other terms	1	$\log_2(3/1) = 1.585$

Final TF-IDF Weights ($TF \times IDF$):

Term	d1	d2	d3
star	$0.167 \times 0.585 = 0.098$	$0.143 \times 0.585 = 0.084$	0
light	$0.167 \times 1.585 = 0.265$	0	0
heat	$0.167 \times 1.585 = 0.265$	0	0
solar	$0.167 \times 1.585 = 0.265$	0	0
system	$0.167 \times 1.585 = 0.265$	0	0
provides	$0.167 \times 1.585 = 0.265$	0	0
Hollywood	0	$0.143 \times 1.585 = 0.227$	0

walked	0	$0.143 \times 1.585 = 0.227$	0
red	0	$0.143 \times 1.585 = 0.227$	0
carpet	0	$0.143 \times 1.585 = 0.227$	0
movie	0	$0.143 \times 1.585 = 0.227$	0
premiere	0	$0.143 \times 1.585 = 0.227$	0
Astronomers	0	0	$0.143 \times 1.585 = 0.227$
observe	0	0	$0.143 \times 1.585 = 0.227$
distant	0	0	$0.143 \times 1.585 = 0.227$
stars	0	0	$0.143 \times 1.585 = 0.227$
galaxies	0	0	$0.143 \times 1.585 = 0.227$
using	0	0	$0.143 \times 1.585 = 0.227$
telescopes	0	0	$0.143 \times 1.585 = 0.227$

Q3: Boolean Retrieval — Query: "star light"

Using the inverted index:

"star" $\rightarrow$ {d1, d2}

"light" $\rightarrow$ {d1}

OR retrieval (any query term matches): {d1, d2}

AND retrieval (all query terms match): {d1}

Documents matched: d1 contains both "star" and "light". d2 contains only "star".

Q4: Cosine Similarity and Ranking — Query: "star light"

Query TF-IDF vector:

Query tokens: {star, light} — each appears once, query length = 2

$$\text{TF}(\text{star}) = 1/2 = 0.5, \text{IDF}(\text{star}) = 0.585 \rightarrow \text{weight} = 0.293$$

$$\text{TF}(\text{light}) = 1/2 = 0.5, \text{IDF}(\text{light}) = 1.585 \rightarrow \text{weight} = 0.793$$

Query vector $q = [\text{star: } 0.293, \text{light: } 0.793]$

Cosine Similarity = $(q \cdot d) / (|q| \times |d|)$:

$$|q| = \sqrt{(0.293^2 + 0.793^2)} = \sqrt{(0.086 + 0.629)} = \sqrt{0.715} = 0.846$$

d1 vector (star: 0.098, light: 0.265):

$$\text{dot} = (0.293 \times 0.098) + (0.793 \times 0.265) = 0.029 + 0.210 = 0.239$$

$$|d1| = \sqrt{(0.098^2 + 0.265^2 + \dots)} \approx \sqrt{(5 \times 0.265^2 + 0.098^2)} = \sqrt{(0.351 + 0.010)} = \sqrt{0.361} = 0.601$$

$$\cos(q, d1) = 0.239 / (0.846 \times 0.601) = 0.239 / 0.508 = 0.470$$

d2 vector (star: 0.084, all others 0 for query terms):

$$\text{dot} = (0.293 \times 0.084) + (0.793 \times 0) = 0.025$$

$$|d2| = \sqrt{(0.084^2 + 6 \times 0.227^2)} = \sqrt{(0.007 + 0.309)} = \sqrt{0.316} = 0.562$$

$$\cos(q, d2) = 0.025 / (0.846 \times 0.562) = 0.025 / 0.476 = 0.052$$

d3 vector (no query terms present):

$$\text{dot} = 0$$

$$\cos(q, d3) = 0.000$$

Final Ranking:

Rank	Document	Score
1	d1	0.470
2	d2	0.052
3	d3	0.000

The ranking is desirable. d1 contains both query terms "star" and "light" and is correctly ranked first with the highest similarity score. d2 contains only "star" and is ranked second. d3 contains neither term and is correctly ranked last. The TF-IDF cosine similarity appropriately rewards documents that match more query terms with higher weights.

Q5: Analysis of Word Sense Ambiguity — Query: "movie star"

Which document should ideally be retrieved?

Ans: d2 should ideally be retrieved as it refers to a Hollywood celebrity ("star" in the entertainment sense) and explicitly mentions "movie premiere."

Does the system retrieve the correct document?

Ans: Yes, d2 is ranked first for this query since it contains both "movie" and "star," giving it the highest cosine similarity score.

How does word sense ambiguity affect retrieval?

Ans: The term "star" is polysemous — it carries at least two distinct meanings:

1. An astronomical object (as in d1: "The star in our solar system")
2. A celebrity or famous person (as in d2: "Hollywood star")

The VSM treats "star" as a single token regardless of meaning. Therefore, d1 also receives a non zero similarity score for the query "movie star" because it contains the token "star," even though d1 is about astronomy and is completely irrelevant to the query's intent. This is a classic example of word sense ambiguity degrading retrieval precision the system cannot distinguish between the astronomical and entertainment senses of "star," leading to potentially irrelevant documents being retrieved alongside relevant ones.

Part 05 :

Q1: Limitations of VSM with Cranfield Examples

1. Synonymy — No Semantic Understanding

VSM matches exact tokens only. Semantically equivalent queries return completely different documents. Example from my system:

- Query "heat transfer" → Top 5 docs: **398, 554, 145, 559, 120**
- Query "thermal conduction" → Top 5 docs: **31, 586, 1178, 95, 1146**

Zero overlap despite both queries referring to the same physical phenomenon. A user searching either phrase should ideally get the same relevant documents.

```
1078
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
heat transfer

Top five document IDs :
398
554
145
559
120
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
thermal conduction

Top five document IDs :
31
586
1178
95
1146
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> |
```

2. Term Independence Assumption

VSM treats all terms independently. Query "turbulent flow" and "laminar flow" share the term "flow" but refer to fundamentally different aerodynamic phenomena. VSM has no mechanism to model the relationship between terms.

- Query "turbulent flow" → Top 5 docs: **151, 99, 1281, 258, 976**
- Query "laminar flow" → Top 5 docs: **1228, 135, 55, 98, 1076**

```

PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
turbulent flow

Top five document IDs :
151
99
1281
258
976
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
laminar flow

Top five document IDs :
1228
135
55
98
1076
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> █

```

3. Corpus Stopword Over-removal

Aggressive corpus-specific stopwords removal eliminated high frequency but semantically critical domain terms such as "flow", "pressure" and "lift" from query vectors. This caused all queries to return identical default rankings (docs 1–5), with Precision@1 = 0.062 and MAP@10 = 0.026. After disabling corpus stopwords, Precision@1 jumped to **0.649** and MAP@10 to **0.303**, confirming that domain-specific frequent terms carry high retrieval value in technical corpora.

Before:

```

PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py
Precision, Recall, F-score @ 1: 0.06222222222222222, 0.011134199134199135, 0.030785634118967446
MAP, nDCG, MRR @ 1: 0.011134199134199135, 0.051111111111111114, 0.06222222222222222
Precision, Recall, F-score @ 2: 0.04222222222222223, 0.015324675324675324, 0.029735345875696748
MAP, nDCG, MRR @ 2: 0.013822029822029824, 0.043107619974273655, 0.06888888888888889
Precision, Recall, F-score @ 3: 0.03407407407407406, 0.017991341991341996, 0.027761769023715452
MAP, nDCG, MRR @ 3: 0.015204745871412538, 0.04030698780654271, 0.07333333333333333
Precision, Recall, F-score @ 4: 0.03333333333333333, 0.02228763828763829, 0.02886661903188728
MAP, nDCG, MRR @ 4: 0.016358185024851694, 0.04013440713032251, 0.08
Precision, Recall, F-score @ 5: 0.034666666666666666, 0.027950937950937955, 0.031477642924962794
MAP, nDCG, MRR @ 5: 0.01753317299983967, 0.04318026756540492, 0.08711111111111111
Precision, Recall, F-score @ 6: 0.035555555555555557, 0.03435305435305436, 0.03390367600283981
MAP, nDCG, MRR @ 6: 0.019256277056277055, 0.045491177405418075, 0.0908148148148148
Precision, Recall, F-score @ 7: 0.03873015873015873, 0.04492817059483725, 0.038189741623222584
MAP, nDCG, MRR @ 7: 0.021271921024301973, 0.05173846860087112, 0.09589417989417987
Precision, Recall, F-score @ 8: 0.04, 0.054490941157607824, 0.040755173001405895
MAP, nDCG, MRR @ 8: 0.0231712115485925, 0.056560936298191325, 0.09867195767195763
Precision, Recall, F-score @ 9: 0.04098765432098762, 0.06339409972743305, 0.04258161070002755
MAP, nDCG, MRR @ 9: 0.02461804318060931, 0.05988978807956596, 0.1016349206349206
Precision, Recall, F-score @ 10: 0.041777777777777754, 0.06933413500080166, 0.04396337209839757
MAP, nDCG, MRR @ 10: 0.025883651646217778, 0.06319749341406762, 0.10296825396825394
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> █

```

After:

```
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py
Precision, Recall, F-score @ 1: 0.6488888888888888, 0.10919181807099311, 0.30341119526435006
MAP, nDCG, MRR @ 1: 0.10919181807099311, 0.5188888888888888, 0.6488888888888888
Precision, Recall, F-score @ 2: 0.5466666666666666, 0.1808846901723276, 0.35817318332622045
MAP, nDCG, MRR @ 2: 0.1728968396537754, 0.480851773712128, 0.6977777777777778
Precision, Recall, F-score @ 3: 0.48592592592592615, 0.228655709398868, 0.36717121379987616
MAP, nDCG, MRR @ 3: 0.21083802090496695, 0.4586421306186382, 0.7140740740740742
Precision, Recall, F-score @ 4: 0.43444444444444447, 0.2645503767399963, 0.3586815802537942
MAP, nDCG, MRR @ 4: 0.23545837908188896, 0.44551456384575994, 0.7207407407407408
Precision, Recall, F-score @ 5: 0.38577777777777794, 0.2883589899754627, 0.3387970885063874
MAP, nDCG, MRR @ 5: 0.24881274079236254, 0.43517543010687126, 0.726962962962963
Precision, Recall, F-score @ 6: 0.35851851851851846, 0.3241227385725446, 0.3308718303061189
MAP, nDCG, MRR @ 6: 0.2650467742486182, 0.4393412623021309, 0.7321481481481483
Precision, Recall, F-score @ 7: 0.33396825396825425, 0.348107742542606, 0.3190122289384945
MAP, nDCG, MRR @ 7: 0.2771464162778169, 0.44188454174874475, 0.7334179894179895
Precision, Recall, F-score @ 8: 0.31666666666666665, 0.3713727081774375, 0.3102304973778975
MAP, nDCG, MRR @ 8: 0.2883490910459071, 0.44888521657621694, 0.7339735449735452
Precision, Recall, F-score @ 9: 0.29975308641975346, 0.3931739889591105, 0.3002991786885825
MAP, nDCG, MRR @ 9: 0.29666185610703816, 0.4552953198502894, 0.7359488536155205
Precision, Recall, F-score @ 10: 0.28355555555555557, 0.40999451439276363, 0.28923360662858977
MAP, nDCG, MRR @ 10: 0.3025772846456973, 0.4589211350766685, 0.7368377425044094
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02>
```

4. No Word Order / Phrase Awareness

"boundary layer separation" and "separation boundary layer" produce identical TF-IDF vectors. Compound technical terms common in Cranfield lose their meaning when treated as independent tokens.

5. High Sparsity

With ~1400 documents and large vocabulary, the TF-IDF matrix is extremely sparse. Most query-document cosine similarities are near zero, causing poor discrimination between relevant and irrelevant documents. Reflected in $\text{Recall}@10 = 0.41$, system still misses ~59% of relevant documents even at $k=10$.

2. What happens when a query contains an OOV term?

In the VSM, each query term is looked up in the index to retrieve its IDF weight. If a term does not exist in the document vocabulary, it has no entry in the inverted index and receives a TF-IDF weight of zero. The query vector is computed only from terms that exist in the vocabulary. OOV terms contribute nothing. If all query terms are OOV, the query vector becomes a zero vector, making cosine similarity undefined for all documents, resulting in a score of zero across the board.

How does this affect retrieval in our system?

When the query vector is empty (all terms OOV), the `rank()` function returns documents in their original index order (IDs 1, 2, 3, 4, 5...) completely unrelated to the query. The user receives

no meaningful results and no warning or error is shown. This is a silent failure the system appears to return results but they are entirely meaningless.

Example query demonstrating this issue:

- Query: **"smartphone cryptocurrency blockchain"**
- Result: Top 5 docs: **1, 2, 3, 4, 5** — default index order, zero relevance to query
- Reason: None of these terms exist in the 1960s aerodynamics Cranfield vocabulary. All three receive zero TF-IDF weight → query vector collapses to zero → cosine similarity is zero for all documents → system falls back to default document order.

```
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
smartphone cryptocurrency blockchain

Top five document IDs :
1
2
3
4
5
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> |
```

Contrast with in-vocabulary query:

- Query: **"aircraft supersonic flow"**
- Result: Top 5 docs: **51, 253, 1169, 100, 884** meaningful, query-relevant ranking.

```
PS C:\Users\aminu\OneDrive\Desktop\NLP Assignment 02> python main.py -custom -dataset cranfield/ -out_folder output/
Enter query below
aircraft supersonic flow

Top five document IDs :
51
253
1169
100
884
```